CASE-STUDY PRACTICE | CAPSTONE 13 | STUDY EDITION

Autonomous Coding Agents: Planning, Execution and the Cost of a Wrong Step

Scenario questions on an agent that resolves a coding task end to end through a plan-and-execute loop, each opening into deeper follow-ups, with answers that explain how to think. Built on the AI System Design Primer case study of the same name.

Jump to the contents | The case in one page | Numbers to know | Decision table | Sources

HOW TO STUDY WITH THIS BOOK

1. Read the amber **concepts box** at the start of a part and look up anything unfamiliar. 2. Read a **scenario** and, before reading further, write down what you would ask and what you would check. 3. Read the green **how to think** box, which names the mental move the question tests. 4. Walk down the nested steps: cover each answer, predict it, then compare. Each answer exposes the next question. The last step of every question asks for the general lesson in one sentence.

The pattern behind most questions

- A number or a technique arrives with authority: a published benchmark, a cost estimate, a shard count, a rate.
- The interviewer asks what it assumes, and what happens when the assumption fails.
- A strong answer names the hidden assumption, puts a number on it, and says how to measure it on your own data.
- The last step usually reframes the original question.

EVIDENCE TAGS

source-confirmed: checked against the named primary source. **illustrative**: this document's planning assumption. **recomputed**: arithmetic re-derived from the stated inputs. **reasoned**: an argument from the design, not tested.

A NOTE ON SOURCES AND ON WHAT WAS CHECKED

The scenarios are written for this document; none comes from a real company question bank. Nothing here was run on a coding-agent system: figures are the sources' own claims or arithmetic from stated inputs, and fleet sizes, round-trip counts, prices, latencies and blast-radius ratios are illustrative. The last page lists the sources and says how much of each was read. A separate Mock Interview edition (answers hidden) uses the same questions and numbering.

Contents

Every line below is a link: click it to jump to that page. The outline panel of your PDF viewer lists the same entries, and the “Contents” link in every page footer brings you back here.

Front matter

A The case in one page 3

B Numbers to know 4

C Decision table 5

Parts at a glance

1 Framing the objective and reading the benchmark 6

2 Sizing the fleet and the cost of a task 10

3 The agent core and the context it reasons over 14

4 Serving, isolation, and operating the agent 18

Sources and confidence 22

Every question, by part

Part 1 Framing the objective and reading the benchmark 6

01 Single-shot or multi-step? Framing 6

02 How does SWE-bench grade? Evaluation 7

03 Two bars for done Evaluation 8

04 Read the 13.86% claim Evaluation 9

Part 2 Sizing the fleet and the cost of a task 10

05 Size the sandbox fleet Sizing 10

06 Cost one task Sizing 11

07 Which input is most sensitive? Sizing 12

08 Cost of one benchmark run Sizing 13

Part 3 The agent core and the context it reasons over 14

09 Why is the codebase not a static corpus? Data 14

10 Why must a long run checkpoint? Data 15

11 Which control-flow pattern fits? Agent design 16

12 Why is per-step correction load-bearing? Agent design 17

Part 4 Serving, isolation, and operating the agent 18

13 Why not one synchronous request? Serving 18

14 Review the retried edit action Code review 19

15 Scope the tools and bound a breach Security 20

16 Catch a stalled run and decide the merge policy Operations 21

The case in one page

A summary of the case study to read before the questions.

Problem	Given a coding task and real access to a codebase, run a sequence of actions to a verified solution, with no human approving each step.
Not single-shot	A completion tool makes one call and stops. This agent loops, observing each result so it can catch and fix its own mistakes mid-task.
Devin's claims	Plan and execute tasks of thousands of decisions; shell, editor and browser in a sandbox; real-time progress reporting (source-confirmed).
The headline	13.86% of SWE-bench resolved end to end, against a prior best of 1.96% and 4.80% for assisted models; a 25% subset, Devin unassisted (self-reported).
Not published	Devin's real architecture, tool-call counts, durations, concurrency and cost. Every sizing figure and the designs in Steps 3 to 5 are this document's.
First to break	Error compounding. A 95%-reliable step run 50 times finishes clean under 8% of the time, so observing and correcting each step is load-bearing.
Easy-to-miss traps	A benchmark score is not a merge-ready PR. 13.86% was a quarter of the data (about 79 tasks), unassisted against assisted, no independent report. The round-trip count drives cost, not the sandbox.

Numbers to know

Each figure carries the tag that says how it was obtained. Cognition's SWE-bench figures are its own launch claim, on a 25% subset. Fleet, token, cost and latency figures are illustrative, because Cognition publishes none.

Quantity	Value	Evidence
Devin's SWE-bench claim	13.86% resolved end to end; prior best 1.96%; 4.80% for assisted models; random 25% subset; Devin unassisted, others assisted	source-confirmed (Cognition, self-reported)
Subset size	25% of 2,294 is about 573 tasks; 13.86% is about 79 tasks resolved	recomputed
Benchmark scale	2,294 real GitHub issues; 12 Python repositories; test-based fail-to-pass and pass-to-pass grading; 1.96% prior best was Claude 2 with BM25	source-confirmed (SWE-bench paper)
Fleet	500 concurrent sessions; 4 vCPU and 8 GB each; 2,000 vCPUs and 4,000 GB reserved	illustrative; recomputed
Task durations	median 20 minutes; p90 90 minutes; p99 240 minutes	illustrative
Tokens per task	200 round-trips; 3,000 input and 300 output each; 0.6M input, 0.06M output	illustrative; recomputed
Cost per task	0.6 x 3 + 0.06 x 15 = 2.70 dollars model; 1.5 h x 0.20 = 0.30 sandbox; about 3.00 dollars	illustrative; recomputed
Sensitivity	at 2,000 round-trips the model cost alone is about 27 dollars; total about 27 dollars or more	illustrative; recomputed
One benchmark run	2,294 x 3 is about 6,882 dollars for a full pass; about 1,700 dollars on the 25% subset	illustrative; recomputed
Compounding error	0.95 to the 50th power is about 0.077, a 7.7% clean-run chance open-loop	illustrative; recomputed
Timeout ratio	5,400 s task against a 60 s ALB idle timeout is 90 to 1	source-confirmed (ALB default); recomputed
Blast radius	tenant-isolated reaches 1; bin-packed 20 co-located reaches 20; 20 to 1	illustrative; recomputed
Sandbox limits	4 vCPU; 8 GB; pids 512; 20 GB disk; timeout 7,200 s (120 minutes)	illustrative

Decision table

What this design chooses, and when to choose otherwise.

Decision	Default here	Choose otherwise when	Evidence
Objective	Multi-step task completion with execution feedback	Single-shot generation for a trivial task	reasoned
Control-flow pattern	Plan-and-execute with per-step observation	Plain ReAct, or an open-loop fixed plan	source-confirmed; reasoned
Context refresh	Reflect the current repo state each step	Index once up front if files never change	reasoned
Checkpointing	Store the full plan array and the last result	A single status field for short tasks	reasoned
Serving	Fast submit, task queue, progress channel	One synchronous request for short tasks	source-confirmed; reasoned
Step idempotency	Detect the already-applied state before an edit	Blind retry for read-only actions only	reasoned
Tool credentials	Three scoped credentials, one per tool	One shared grant if the blast radius is acceptable	source-confirmed; reasoned
Tenant isolation	One tenant per sandbox	Bin-pack if the escape risk is acceptable	reasoned
Merge policy	Human review where a wrong merge is costly	Auto-merge on CI where it is cheap	reasoned

PART 1

Framing the objective and reading the benchmark

Separating a multi-step agent from a single-shot generator, defining how SWE-bench grades, and reading a launch number for what it actually says.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Autonomous coding agent	A system that, given a coding task and real access to a codebase, decides and runs a sequence of actions, reads files, runs shell commands, edits code, runs tests, and continues to a verified solution without a human approving each step.
Single-shot generation	One model call that produces one artifact and stops, with no execution or test-running step to catch a wrong answer.
Plan-and-execute	A control-flow pattern where the model produces a full multi-step plan up front, then executes each step in order, optionally revising the plan when a step's result changes what is needed.
SWE-bench	A benchmark of 2,294 real GitHub issues from 12 open-source Python repositories, each paired with the human pull request that originally fixed it. The model sees the issue and the repository just before the fix, and must produce its own patch.
Resolve rate	The share of a benchmark's tasks a model's patch resolves under the benchmark's own grading rule.
Fail-to-pass and pass-to-pass	SWE-bench grades by tests: fail-to-pass tests were failing before the fix and must now pass; pass-to-pass tests were already passing and must stay passing. One regression fails the whole task.
Two bars for done	The benchmark bar is a test-based pass or fail. The real-world bar is a pull request that passes CI and a human reviewer accepts. The second is stricter.
Self-reported figure	A number published by the party that benefits from it, often at launch and before an independent report. It is evidence, not an audit.
Where to read	Cognition Labs: Introducing Devin (March 2024) | Jimenez et al.: SWE-bench, Can Language Models Resolve Real-World GitHub Issues? (ICLR 2024, arXiv:2310.06770)

01 Single-shot or multi-step?

FRAMING

SCENARIO

A product manager says the coding agent is just a code-completion model with a nicer prompt, so one model call per task should do. Explain why the task in this case study is a different shape.

HOW TO THINK ABOUT IT

Ask what a single model call can and cannot observe, and what happens when it is wrong.

Q What does a single-shot tool do?

A One prompt in, one artifact out, in one model call. Nothing inside the tool runs a test or observes a result, so if the output is wrong nothing catches it.

- It is a generator, not a loop.

Q What does this agent do instead?

A It decides a sequence of actions, reads files, runs commands, edits code, runs tests, observes the result of each, and continues until the solution is verified, with no human approving each step (source-confirmed, the post claims 'thousands of decisions').

- The feedback from running a test is the part a generator never gets.

Q Why does the difference matter for failure?

A A generator is right or wrong in one call's worth of risk. A multi-step agent can go wrong at any of many decisions, and those errors compound across steps (Part 3 makes this concrete).

- Larger surface, but with a chance to catch and fix mistakes mid-task.

Q What is being optimised?

A End-to-end completion across many sequential decisions, not the quality of any one generated artifact.

- The metric is task resolution, not snippet quality.

Q Reframe it.

A Decide first whether the objective is one output or a completed task. That choice sets the whole architecture: a loop that observes and corrects, not a single call.

Basis: The 'thousands of decisions' and 'fix mistakes' framing is Cognition's own (source-confirmed); the compounding argument is this document's (reasoned).

02 How does SWE-bench grade?

EVALUATION

SCENARIO

A teammate says the agent 'passed' a bug because its diff looked correct on inspection. Explain how SWE-bench actually decides whether a task is resolved, and why that rule is used.

HOW TO THINK ABOUT IT

Describe what SWE-bench contains, then its grading criterion, and why it is test-based rather than a judgment.

Q What is in the benchmark?

A 2,294 real GitHub issues from 12 popular open-source Python repositories, each paired with the human pull request that originally fixed it. The model gets the issue and the repo at the commit before the fix (source-confirmed, SWE-bench paper).

- It is drawn from real projects, not synthetic tasks.

Q How is a task scored resolved?

A The patch is applied and the repository's real test suite is run. The task resolves only if every fail-to-pass test now passes and no pass-to-pass test breaks. One regression anywhere fails the whole task.

- Grading is deterministic and test-based, not a human or model opinion.

Q Why grade by tests, not by looking?

A A model's own claim that it fixed a bug means nothing without an objective, automatable check. Code correctness, unlike a free-text answer, has a checkable definition.

- This is why the agent can self-evaluate at all, by running tests.

Q What does resolve rate not capture?

A Whether the fix addresses the issue's real intent rather than its symptom, its style, and risk the existing tests do not cover. A patch can pass every test and still be a fix a reviewer rejects.

- That gap is Q3's second bar.

Q Reframe it.

A State the grading rule before quoting a score. 'Resolved' here means a specific test outcome, not 'looks right'.

Basis: The benchmark scale and grading rule are from the SWE-bench paper (source-confirmed); the intent-versus-symptom gap is reasoned.

03 Two bars for done

EVALUATION

SCENARIO

An engineer wants to ship any patch that clears SWE-bench's grading. Explain the second, stricter bar and why the benchmark is only a proxy for it.

HOW TO THINK ABOUT IT

Name the two definitions of done and say how they differ.

Q What is the benchmark bar?

A A test-based pass or fail: fail-to-pass tests pass, pass-to-pass tests stay passing. It needs no human per run, so it scales.

- Cheap, reproducible, automatable.

Q What is the real-world bar?

A A pull request that passes CI and a human reviewer accepts. The reviewer also judges intent, style and uncovered risk.

- This is what a team actually merges.

Q Why is resolve rate only a proxy?

A It is strictly narrower: Python only, issue-shaped tasks only, no design work, and no review of the fix's quality beyond the named tests. A test-passing patch can still be the wrong fix.

- A proxy is cheaper and scalable, but it is not the target.

Q How should the two be used together?

A Resolve rate for fast, repeatable offline iteration; the CI-plus-review bar as the real acceptance gate and the source of production feedback (Part 4).

- Track both; do not let the cheap one stand in for the real one.

Q Reframe it.

A When a benchmark is a proxy, say so and name the stricter target it stands in for. Shipping to the proxy optimises the wrong thing.

Basis: Both bars are this document's framing built on the sources (reasoned); the benchmark's narrowness follows from its own definition (source-confirmed).

04 Read the 13.86% claim

EVALUATION

SCENARIO

A slide quotes: Devin resolves 13.86% of SWE-bench issues, far above the prior best of 1.96%, and 4.80% for models given the files to edit. A reviewer treats it as an independently verified result. Say what the number does and does not support.

HOW TO THINK ABOUT IT

Read the caveats on the figure, do the subset arithmetic, and ask who reported it.

Q What are the two stated caveats?

A The evaluation ran on a random 25% subset of the dataset, not all 2,294 tasks; and Devin was unassisted while all other models were assisted, meaning the baselines were told exactly which files to edit (source-confirmed, both quoted from the post).

- The asterisk on the headline number carries these.

Q What does the subset mean in tasks?

A 25% of 2,294 is about 573 tasks, so 13.86% is roughly 79 tasks resolved. The number rests on a few hundred tasks, not the full benchmark (recomputed).

- A smaller sample means a wider confidence interval, unstated here.

Q Which way does unassisted-versus-assisted cut?

A It cuts in favour of the gap being real, since Devin had to locate the files itself while the baselines were handed them. But it also means the figures were not measured under identical framing, so it is not apples to apples.

- Harder task for Devin's side, so the reported lead is not inflated by that.

Q Who reported it, and what is missing?

A Cognition, in its own launch post, which said a technical report was still forthcoming. No independent replication or Cognition technical report could be confirmed. Read 13.86% as a self-reported launch claim (source-confirmed, self-reported).

- The prior best of 1.96% was Claude 2 with BM25 retrieval, per the SWE-bench paper's leaderboard.

Q Reframe it.

A Quote a launch number with its sample size, its comparison conditions and its author. A figure on a quarter of the data, self-reported, is a signal, not a verified fact.

Basis: The figures and caveats are Cognition's own, quoted from the post (source-confirmed, self-reported); the task counts are arithmetic (recomputed); the 1.96% attribution is from the SWE-bench paper (source-confirmed).

PART 2

Sizing the fleet and the cost of a task

Applying the back-of-envelope method to a fleet of long-running sessions, costing one task, and naming the input the whole estimate depends on.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Back-of-envelope method	State assumptions, decompose into terms, compute carrying units, sanity-check the order of magnitude, and name the input the answer is most sensitive to.
Round-trip	One model reasoning or tool-call exchange: a request to the model and its response, often followed by running a tool and feeding the result back.
p90 and p99	The 90th and 99th percentile: 90%, respectively 99%, of sessions finish faster than this duration. They describe the tail, not the typical case.
Token pricing anchor	This repo's shared illustrative rate for every cost estimate: 3 dollars per million input tokens and 15 dollars per million output tokens. Chosen for consistency, not as any provider's real price.
Order-of-magnitude check	A sanity test that asks whether a result is in the right ballpark, catching an error of a factor of ten or more, not confirming an exact figure.
Sensitivity	How much the answer moves when one assumption changes. The most sensitive input is where a prototype should measure first.
Where to read	Cognition Labs: Introducing Devin (March 2024) | Jimenez et al.: SWE-bench, Can Language Models Resolve Real-World GitHub Issues? (ICLR 2024, arXiv:2310.06770)

05 Size the sandbox fleet

SIZING

SCENARIO

You run 500 concurrent agent sessions, each in a sandbox budgeted at 4 vCPUs, 8 GB RAM and 20 GB disk. Estimate the fleet-wide compute reserved, and say whether it is plausible.

HOW TO THINK ABOUT IT

Multiply per-session resources by the session count, carrying units, then sanity-check the magnitude.

Q What is the vCPU total?

A 500 sessions times 4 vCPU each is 2,000 vCPUs reserved fleet-wide (illustrative inputs, recomputed).

- Carry the unit: sessions times vCPU per session gives vCPUs.

Q What is the memory total?

A 500 times 8 GB is 4,000 GB of RAM reserved.

- Same shape, different per-session figure.

Q Is that plausible?

A Yes. 2,000 vCPUs and 4,000 GB is the order of a mid-size production cluster, reasonable for hundreds of concurrent sessions. Not off by orders of magnitude either way.

- The check catches a factor-of-ten error, not a small one.

Q Why is most of it idle?

A During the many round-trips where the sandbox waits on a model response, nothing is executing. A generous per-session budget protects a heavy build or test suite but sits idle most of the time.

- Reserved is not the same as used; this is the safety-margin-versus-cost trade-off.

Q Reframe it.

A Size reserved capacity by multiplying per-unit cost by the count, then ask whether the total is a plausible cluster. Flag how much is reserved against idle waiting.

Basis: The 500 sessions and 4 vCPU / 8 GB budget are illustrative; the totals are arithmetic (recomputed).

06 Cost one task

SIZING

SCENARIO

A p90 task runs 90 minutes and takes about 200 round-trips, each 3,000 input and 300 output tokens, at 3 dollars per million input and 15 dollars per million output. Sandbox compute is 0.20 dollars an hour. Estimate the cost of one task.

HOW TO THINK ABOUT IT

Compute the model cost and the sandbox cost separately, carrying units, then add.

Q What are the token totals?

A 200 round-trips times 3,000 input is 600,000 input tokens, 0.6 million. Times 300 output is 60,000 output tokens, 0.06 million (recomputed).

- Keep input and output separate; they are priced differently.

Q What is the model cost?

A 0.6 times 3 is 1.80 dollars of input, plus 0.06 times 15 is 0.90 dollars of output, so 2.70 dollars.

- Output tokens cost five times input here, so they matter despite being fewer.

Q What is the sandbox cost?

A 90 minutes is 1.5 hours, times 0.20 dollars an hour is 0.30 dollars.

- Convert minutes to hours before multiplying.

Q What is the total, and which term dominates?

A About 3.00 dollars per task. The model cost of 2.70 dominates the 0.30 of sandbox compute by nine to one.

- The model, not the sandbox, is where the money goes.

Q Reframe it.

A Cost each term in its own units, then compare their sizes. Knowing the model term dominates tells you which assumption to challenge next.

Basis: The round-trip count, token counts, prices and sandbox rate are illustrative; the costs are arithmetic (recomputed).

07 Which input is most sensitive?

SIZING

SCENARIO

Your 3-dollar-per-task figure rests on 200 round-trips per task, but the source only says a task can require 'thousands of decisions'. Say which input the estimate is most sensitive to and what happens if it is wrong.

HOW TO THINK ABOUT IT

Ask which assumption, if changed, moves the total the most, and recompute at a plausible alternative.

Q Which input dominates the uncertainty?

A The 200 round-trips per task. It is the least grounded number, since the post gives no tool-call counts, only 'thousands of decisions' (source-confirmed for the qualitative claim).

- The token and price figures are anchors; the round-trip count is a guess.

Q What if it is ten times higher?

A At 2,000 round-trips the model cost alone rises to about 27 dollars, swamping the 0.30-dollar sandbox term by nearly two orders of magnitude and pushing total cost from about 3 to 27 dollars or more (recomputed).

- 2,000 round-trips is entirely consistent with 'thousands of decisions'.

Q Why not the sandbox budget?

A The sandbox term is 0.30 dollars and changes little; even doubling it barely moves the total. The round-trip count scales the dominant term linearly.

- Sensitivity follows the dominant term, not the one that feels concrete.

Q What should a prototype measure first?

A The real round-trip count per task. That single number, not the sandbox rate, decides whether a task costs 3 dollars or 30.

- Measure the most sensitive input before optimising anything else.

Q Reframe it.

A Name the input the answer is most sensitive to, recompute at a plausible alternative, and send a prototype to measure that one first.

Basis: The 'thousands of decisions' claim is the source's (source-confirmed); the 200 and 2,000 round-trip figures and their costs are illustrative arithmetic (recomputed).

08 Cost of one benchmark run

SIZING

SCENARIO

A reviewer asks how much it costs to run the full SWE-bench once, and someone guesses 'a few dollars'. Estimate it at this document's own per-task figure, and use the result as an order-of-magnitude check.

HOW TO THINK ABOUT IT

Multiply the task count by the per-task cost, then ask whether the result is in the right ballpark.

Q What is the arithmetic?

A 2,294 tasks times about 3 dollars each is about 6,882 dollars for one full pass (recomputed).

- Use the full 2,294, not the 25% subset, for a full run.

Q Is 'a few dollars' plausible?

A No. The guess is off by three orders of magnitude. A full pass is thousands of dollars, not single digits.

- The check exists to catch exactly that size of error.

Q Is 7 million dollars plausible?

A No, equally wrong the other way. The job of the check is to land between 7 dollars and 7 million, near 7 thousand.

- Right order of magnitude is the goal, not an exact figure.

Q What does this not claim?

A It is not Cognition's real spend, which is unpublished, and the launch run used a 25% subset, so about 1,700 dollars at this rate.

- A sanity check is about magnitude, not about a company's actual bill.

Q Reframe it.

A Use a rough cost to bound a magnitude, and reject a guess that is off by orders of magnitude in either direction, without pretending it is the real number.

Basis: The task count is from the SWE-bench paper (source-confirmed); the per-task cost is illustrative and the run cost is arithmetic (recomputed).

PART 3

The agent core and the context it reasons over

Why the codebase is a corpus the agent keeps changing, why a long run must checkpoint, which control-flow pattern fits, and why per-step correction is not optional.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Self-mutating corpus	The agent's context is the repository's files, and its own edits and test runs change them turn over turn. Unlike a static document corpus, each step must reflect the current state.
State and checkpointing	Saving a long run's progress at defined points so a crash resumes from the last checkpoint instead of restarting the whole task.
ReAct	A control-flow pattern of one step at a time: reason, act, observe, repeat, with no full plan committed up front.
Reflection	Observing the result of a step and adjusting during execution, rather than running a fixed plan blindly to the end.
Open-loop versus closed-loop	Open-loop executes a fixed plan with no checking between steps; closed-loop observes each result and can correct. This agent is closed-loop.
Compounding error	If each of n independent steps is correct with probability p, the chance all n are correct is p to the power n, which falls fast as n grows.
Where to read	Cognition Labs: Introducing Devin (March 2024) | Jimenez et al.: SWE-bench, Can Language Models Resolve Real-World GitHub Issues? (ICLR 2024, arXiv:2310.06770)

09 Why is the codebase not a static corpus?

DATA

SCENARIO

A colleague wants to index the repository once at the start of a task and reuse that index for every planning step, as a RAG chatbot indexes a document corpus. Explain why that breaks here.

HOW TO THINK ABOUT IT

Compare what changes the corpus in a RAG query against what changes it here.

Q What is different from RAG?

A A RAG corpus is largely static during a query; the query does not change it. Here the corpus is the repository's files, and the agent mutates them with its own edits and test runs, turn over turn.

- The agent is both the reader and the writer of its own context.

Q What goes wrong with a one-time index?

A After the agent edits a file, a stale index describes code that no longer exists. The next step reasons about the repository's state when the task started, not its current state.

- It plans against a file it already changed.

Q What does correct behaviour require?

A Each step's context must reflect the repository's current state: re-read or incrementally update the files a step touched, so the plan reasons about what is actually there now.

- Current state, not start-of-task state.

Q What is the cheaper middle path?

A Update only the files a step's action changed, rather than re-indexing everything each step. It is cheaper but relies on the action log being complete; a missed action leaves a stale belief.

- Incremental update trades completeness risk for cost.

Q Reframe it.

A Ask whether the retrieval source changes during the task. If the agent edits its own corpus, context must be refreshed per step, not fetched once.

Basis: The RAG contrast and the per-step refresh are this document's design (reasoned); the self-mutation is inherent to a coding agent.

10 Why must a long run checkpoint?

DATA

SCENARIO

A 90-minute session crashes at minute 70. A colleague's design restarts it from minute 0. Explain why checkpointing is a structural requirement here, not a nice-to-have, and what the session should store.

HOW TO THINK ABOUT IT

Ask what restarting costs once a task runs long enough to realistically fail partway, then what state makes resuming possible.

Q What does restarting from zero cost?

A It re-runs every already-applied edit and already-passed test from minute 0, which is slow and risks re-applying an edit that was already made.

- The longer the task, the more progress is thrown away on any failure.

Q Why is this structural, not optional?

A Once a session can run 90 minutes or more, it will sometimes fail partway. Checkpointing is what makes it resilient instead of losing all progress on any interruption.

- Long-running plus crash-prone forces checkpointing.

Q What state makes resume possible?

A A session record: an id, the repo, the issue, a status, the full plan array with each step's status, the current step index, and the last test result, plus the sandbox id and a duration cap.

- Storing the full plan, not just a single status, is what makes progress auditable and resumable.

Q What does storing the whole plan cost?

A Session-state size grows with plan length, a real if small storage and transfer cost a single-status design would not have.

- Auditability and resumability are bought with that size.

Q Reframe it.

A When a task runs long enough to fail partway, design the checkpoint first. Store enough state, the full plan and the last result, to resume rather than restart.

Basis: The crash scenario and session record are this document's design (reasoned); the field set matches the README's session-state record.

11 Which control-flow pattern fits?

AGENT DESIGN

SCENARIO

An interviewer asks why the agent uses plan-and-execute with per-step observation rather than a plain ReAct loop or a single plan run to the end. Give the trade-off against both.

HOW TO THINK ABOUT IT

State what each alternative gives and gives up, then why the hybrid fits this task shape.

Q Why not single-shot?

A One call, one diff, no execution step. Cheapest and fastest, but it never runs a test or observes a result and cannot recover from being wrong about the codebase. It is the baseline, not a contender.

- It cannot do the task this capstone is scoped to.

Q Why not plain ReAct?

A One step at a time is flexible moment to moment, with no stale plan to revise. But it has no up-front artifact to report as 'here is my plan', and no single place to check whether the run is on track, only a trailing history.

- It gives up auditability for flexibility.

Q Why plan-and-execute with observation?

A A real up-front plan is the artifact progress reports against and a reviewer can inspect mid-run to judge the approach before it finishes. Per-step observation lets it catch and fix mistakes.

- It keeps the plan's auditability and ReAct's ability to react.

Q What does the hybrid cost?

A A plan can become wrong once execution reveals something unanticipated, so the agent must be able to revise it. A plan-and-execute agent that cannot revise is strictly worse than ReAct at surprises.

- Revision logic is the price of the up-front plan.

Q Reframe it.

A Pick the control-flow pattern from the task shape. Multi-hour, many-decision, execution-dependent work wants an auditable plan plus per-step correction, not one extreme.

Basis: The pattern definitions are from Lesson 1 (source-confirmed within the repo); mapping Devin onto the hybrid is this document's reasoned categorisation.

12 Why is per-step correction load-bearing?

AGENT DESIGN

SCENARIO

Suppose each decision is correct with probability 0.95 and a task has 50 decisions, executed open-loop with no checking in between. Estimate the chance of a clean run, and say what it implies.

HOW TO THINK ABOUT IT

Compute p to the power n, then interpret it against the source's own task scale.

Q What is the formula?

A With no correction, the chance all n steps are right is p to the power n, here 0.95 to the 50th power (reasoned model).

- Each step independent and uncorrected is the key assumption.

Q What is the value?

A 0.95 to the 50th power is about 0.077, so roughly a 7.7% chance of a fully clean run (recomputed).

- A strong 95% per-step rate still fails the task over nine times in ten.

Q What happens at the source's scale?

A The post claims tasks can need 'thousands of decisions', not 50. At n in the thousands, p to the power n is essentially zero. Open-loop long autonomy is unachievable at any plausible per-step rate.

- This is illustrative, not a derivation of the real 13.86%.

Q What does that justify?

A The 'fix mistakes' half of the agent, observing each result and correcting, is not a nicety. It is what keeps the success probability from collapsing as steps multiply.

- Closed-loop beats open-loop by cutting the chain of must-all-be-right steps.

Q Reframe it.

A When success needs many steps all correct, compute p to the power n to show why blind execution fails, and why observation and correction are structural.

Basis: The per-step rate and step count are illustrative; p to the power n is arithmetic (recomputed); the 'thousands of decisions' scale is the source's (source-confirmed).

PART 4

Serving, isolation, and operating the agent

Why a long task cannot be one request, making a retried edit safe, scoping each tool's credential, bounding a sandbox escape, and catching a stalled run.

READ THESE FIRST: CONCEPTS COVERED IN THIS PART

Idle timeout	The longest a load balancer or gateway lets one request wait for a response before closing it. AWS ALB defaults to 60 seconds.
Task queue	A queue of jobs, each with a status, retries and a result, run by a worker pool. One job here is one agent step or one whole session.
Idempotency	A property where running an action again has no further effect. Needed because at-least-once delivery and retries can run a job more than once.
Least privilege	Granting each component only the access it needs. Each tool, shell, editor, browser, gets its own scoped credential, not one shared grant.
Blast radius	How much a single failure or breach can reach. Co-locating tenants widens it; isolating them narrows it.
Multi-tenant isolation	Keeping many customers' data and credentials from reaching each other on shared infrastructure.
Stall detection	A rule that flags a session making no real progress, distinct from one that has failed, so a human can intervene before the hard timeout.
Where to read	Cognition Labs: Introducing Devin (March 2024) | Jimenez et al.: SWE-bench, Can Language Models Resolve Real-World GitHub Issues? (ICLR 2024, arXiv:2310.06770)

13 Why not one synchronous request?

SERVING

SCENARIO

A colleague wants the submit call to hold the HTTP connection open until the 90-minute task finishes and return the result. Explain why that cannot work and what the shape should be.

HOW TO THINK ABOUT IT

Compare the task duration against a gateway's idle timeout, then give the three-part decomposition.

Q What is the timeout problem?

A A 90-minute task is 5,400 seconds. Against AWS ALB's 60-second default idle timeout that is a ratio of 90 to 1 (recomputed). The gateway closes the connection long before the task finishes.

- The client and backend cannot override the gateway's timeout.

Q What is the submit shape instead?

A A REST call that submits the task and returns 202 Accepted with a task id in milliseconds, well under any gateway timeout. The work runs elsewhere.

- Accept fast, do the work out of band.

Q Where does the work run?

A In a task queue: one entry is one session, pulled by an orchestrator worker that runs the plan-and-execute loop inside a per-session sandbox, decoupled from the submit request.

- The 90 minutes live in the worker, not the HTTP call.

Q How does the client follow progress?

A A third channel: the client polls a status url or subscribes to a stream or webhook, and receives the session-state record. That record doubles as internal state and progress payload.

- Submit, queue, progress channel: three parts.

Q Reframe it.

A When a task outlives a gateway timeout, split it into a fast submit, a queued background job, and a separate progress channel, not one held-open request.

Basis: The ALB timeout is documented and the ratio is arithmetic (recomputed); the decomposition is this repo's standard async pattern (reasoned).

14 Review the retried edit action

CODE REVIEW

SCENARIO

Each agent step runs as a task-queue job retried up to max_attempts. The editor action below applies a patch. In production, a retried edit sometimes fails outright or corrupts a file. Find the faults and fix them.

```
def apply_edit(job):
    # one agent step; retried up to job["max_attempts"] on failure
    patch_path = write_patch(job["patch"])
    run(["git", "apply", patch_path], check=True)   # same path as a shell retry
    job["status"] = "succeeded"
    return job
```

HOW TO THINK ABOUT IT

Walk a retry after the patch already applied, and compare the editor case against a shell retry.

Q Why is a retried edit unsafe?

A Applying a patch is not idempotent. If the first attempt applied the patch and then the job was retried, after a crash or a timeout, the second 'git apply' runs against an already-patched file and fails or corrupts it.

- Unlike a test command, a patch changes state, so repeating it is not free.

Q Why is this different from a shell retry?

A A shell test command is usually safe to retry, running the same suite twice corrupts nothing. The editor action is treated the same here, with no check of the file's current state.

- The retry logic must branch by tool, not apply uniformly.

Q What should happen before reapplying?

A Check whether the patch is already present. If a reverse apply would succeed, the patch is already there, so mark the step succeeded and skip. Otherwise apply, ideally with a three-way merge so a retry is more robust.

- Detect the already-applied state instead of blindly reapplying.

Q State the fix.

A Before applying, run a reverse-check: if it succeeds the patch is applied, so return success. Otherwise apply with a three-way merge. Verify by hand with a retry after a forced failure between apply and status write.

- The claim-then-act shape from the task queue applies: record intent, make the action repeatable safely.

Q Reframe it.

A Classify each tool action by whether repeating it is safe. For state-changing actions, detect the already-done state before redoing them; do not share retry logic with read-only actions.

REFERENCE CODE

```
def apply_edit(job):
    patch_path = write_patch(job["patch"])
    # already applied? a reverse-check succeeds only if the patch is present
    present = run(["git", "apply", "--reverse", "--check", patch_path])
    if present.returncode == 0:
        job["status"] = "succeeded"                # idempotent: nothing to redo
        return job
    run(["git", "apply", "--3way", patch_path], check=True)  # safer on retry
    job["status"] = "succeeded"
    return job
```

Basis: The fault is a standard idempotency error and the fix was reasoned and checked by hand, not run; the sources do not describe an editor implementation (reasoned).

15 Scope the tools and bound a breach

SECURITY

SCENARIO

A colleague grants the agent one 'full dev environment' credential for shell, editor and browser together. Explain the least-privilege design instead, and quantify what co-locating tenants does to a sandbox escape.

HOW TO THINK ABOUT IT

Scope each tool to what it needs, then compute the blast radius under isolation against bin-packing.

Q How should the three tools be scoped?

A Separately. The browser tool reaches an allowlisted network but has no filesystem and cannot run shell commands. The shell and editor tools touch the workspace but have no open network, since neither needs it.

- A browser tool has no legitimate need for filesystem-delete access.

Q Why scope per tool?

A If one tool's credential leaks or is hijacked, for example by prompt injection through a malicious page the browser reads, the blast radius is bounded to that one tool's scope, not the full environment.

- One grant means one breach reaches everything; three grants bound each.

Q What does co-location do to an escape?

A Under tenant isolation, one customer per sandbox, an escape reaches one tenant. Bin-packed with 20 tenants on one host sharing a kernel, the same escape reaches all 20: a ratio of 20 to 1 (recomputed).

- The vulnerability is identical; only the ceiling on what it reaches changed.

Q Why does this raise the stakes beyond a basic sandbox?

A A coding sandbox holds real source code with real secrets, API keys, database credentials, CI tokens, for up to the 240-minute p99, on a platform serving many customers' repos at once. It combines single-run and multi-tenant isolation.

- Higher value held longer, across more tenants, than a stateless tool call.

Q Reframe it.

A Scope every tool to least privilege and decide co-location deliberately: density buys efficiency and multiplies the blast radius of any one escape.

Basis: The per-tool scoping follows Lesson 7 (source-confirmed within the repo); the 20-to-1 ratio is illustrative arithmetic (recomputed).

16 Catch a stalled run and decide the merge policy

OPERATIONS

SCENARIO

An agent edits the wrong file for an hour while every step 'succeeds' by its own account, then opens a PR that passes CI. Design the detection and say how to decide whether to merge.

HOW TO THINK ABOUT IT

Separate a stalled session from a failing one, then weigh the two merge extremes.

Q Why do timeouts alone miss this?

A A hard duration cap stops a runaway eventually, but a session can keep reporting success while making no real progress, repeatedly editing the wrong file, or a flaky test bouncing the last result between pass and fail.

- Succeeding by its own account is not the same as progressing.

Q What detects a stall?

A A rule flagging no advancement of the plan step index across K consecutive sandbox-tool-call cycles catches a stalled session well before its hard timeout, distinct from a failing one.

- Watch progress of the plan index, not just liveness.

Q What does a flagged session escalate to?

A A human intervention path: pause, redirect or cancel, for example a delete endpoint that terminates the sandbox and marks the session cancelled, plus an approval gate before high-risk steps such as a remote push.

- Detection is useful only if it routes to an action.

Q How to decide the merge policy?

A Full human review is safest against the real-world bar but does not scale once review is the bottleneck. Auto-merge on CI is faster but risks a test-passing wrong fix. The choice depends on how costly a wrong-but-passing merge is for that codebase.

- A business decision the architecture does not make for you.

Q Reframe it.

A Monitor real progress, not self-reported success, route stalls to a human, and set the merge gate by the cost of a wrong-but-passing merge, not by convenience.

Basis: The stall rule and intervention path are this document's design matching the README (reasoned); the test-only-versus-review gap follows from Part 1's two bars.

Sources and confidence

The case study this book is built on is the AI System Design Primer's Capstone #13. The primary sources are Cognition Labs' 'Introducing Devin' (March 2024), read in full from the live page, and the SWE-bench paper (Jimenez et al., ICLR 2024), read for the benchmark's own definition. Devin's 13.86% / 1.96% / 4.80% figures are Cognition's own launch claim, on a random 25% subset, with Devin unassisted and the baselines assisted, and no independent technical report could be confirmed. The agent architecture in Steps 3 to 5 is a teaching design built on this repo's lessons, not a documented account of Devin's internals.

- Cognition Labs: Introducing Devin, March 2024, read in full from the live page 2026-10-17
https://cognition.ai/blog/introducing-devin

- Jimenez et al.: SWE-bench, Can Language Models Resolve Real-World GitHub Issues?, ICLR 2024 (arXiv:2310.06770), read 2026-10-17
https://arxiv.org/abs/2310.06770